

03_Drag_Calculation: Coastdown-Based Estimation of CdA and Cr

Validation_Aero_Test

February 12, 2026

1 Overview

This document describes the current drag and rolling-resistance estimation implemented in `Code/03_Drag_Calculation/`.

The pipeline assumes that straight-line coastdown intervals have already been extracted (Step 02) and that per-run CSVs exist in `Outputs/02_Run_Extraction/csv/`.

2 Inputs

- Intervals: `Outputs/02_Run_Extraction/csv/drag_intervals.csv` (fields include `run_id`, `t0_s`, `t1_s`)
- Per-run time series: `Outputs/02_Run_Extraction/csv/run_{XXX}.csv`
- Vehicle + analysis constants: `Data/vehicle_constants.json`

The drag calculation requires (per-run CSV columns):

- `t_s` [s]
- `GPS1_VEL_MAG_ms` [m s^{-1}]
- `IMU_ACCEL_LONG_ms2` [m s^{-2}]

3 Preprocessing per interval

Preprocessing is implemented in `Code/03_Drag_Calculation/common.py` as `prepare_run_slice`. Given a per-run CSV and an interval $[t_0, t_1]$:

1. Slice to $t \in [t_0, t_1]$.
2. (Best effort) enforce coastdown conditions if available: `throttle ICU_Apps_pct` $\leq 1\%$, `brakes` ≤ 0.1 bar.
3. Discard low-speed samples: $v \geq v_{\min}$ (default 5 m/s).
4. Define positive deceleration as $a_{\text{dec}} = -a_{\text{long}}$.
5. Smooth a_{dec} with a centered rolling median (default window 7 samples).
6. Construct regression features v and v^2 .

4 Model, fitting, and conversions

Coastdown model

Neglecting grade and wind, the coastdown dynamics can be approximated by

$$m \dot{v} = -\frac{1}{2} \rho C_d A v^2 - C_r m g.$$

Using the positive deceleration convention $a_{\text{dec}} = -\dot{v}$, the maintained pipeline fits the baseline model

$$a_{\text{dec}}(v) = k_2 v^2 + c.$$

The conversions are:

$$C_d A = \frac{2mk_2}{\rho}, \quad C_r = \frac{c}{g}.$$

If a frontal area A is provided per configuration, then

$$C_d = \frac{C_d A}{A}.$$

Forces at a given speed v :

$$F_d(v) = \frac{1}{2} \rho C_d A v^2, \quad F_{rr} = C_r m g.$$

With the positive deceleration convention, the fitted model implies

$$m a_{\text{dec}}(v) = F_d(v) + F_{rr}.$$

Robust regression

Two robust fitting helpers are used:

- `robust_linear_fit`: iterative 2σ clipping for the 1D model $y = mx + b$.
- `robust_linear_fit_multi`: iterative 2σ clipping for a multivariate linear model $y = X\beta$.

Both perform an initial least-squares fit, compute residuals, remove outliers exceeding a threshold, and refit for a fixed number of iterations.

Vehicle constants and configuration mapping

Constants are loaded from `Data/vehicle_constants.json` via `load_constants`. Key fields:

- `vehicle.mass_kg`, `vehicle.rho_kgm3`
- `vehicle.frontal_area_m2_by_config` (optional, for reporting C_d)
- `analysis.fd_speeds_ms` (speeds for reporting drag force table)
- `analysis.baseline_config` (reference used for $\Delta C_d A$ and component contributions)
- `analysis.config_ranges_by_run_id` mapping config name $\rightarrow$ [start, end]

If no mapping is provided, the code falls back to a conventional default (`full`, `no_rear`, `no_front_rear`, `no_front`).

5 Implemented approaches (with example plots)

Each approach is a standalone CLI script that writes a self-contained output folder under `Outputs/03_Drag_Calculation/`.

5.1 Approach 1: Per-run (baseline)

Code/03_Drag_Calculation/Approach1_PerRun/run.py

- Fit $a_{\text{dec}} = k_2 v^2 + c$ per extracted interval.
- Convert k_2, c to $C_d A$ and C_r , then summarize per configuration.
- Emit per-run fit plots for QC.

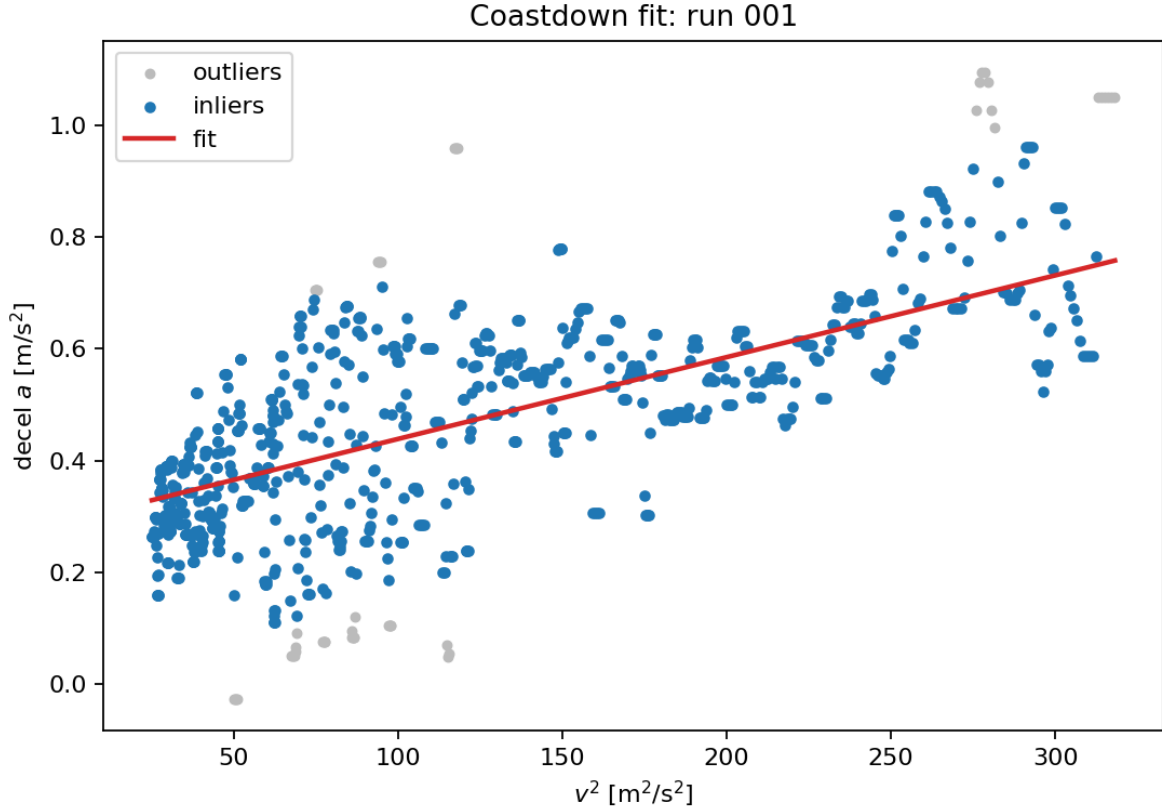


Figure 1: Approach 1: example per-run coastdown fit plot (Run 001).

5.2 Approach 2: Paired opposites (baseline)

Code/03_Drag_Calculation/Approach2_PairedOpposites/run.py

- Fit per run (as in Approach 1).
- Pair consecutive run IDs ($i, i+1$) within a configuration and average coefficients.
- Motivation: reduce steady biases that change sign with direction (e.g., wind/grade) when the experiment is executed as out-and-back passes.

5.3 Approach 3: Pooled by configuration (baseline)

Code/03_Drag_Calculation/Approach3_PooledByConfig/run.py

- Stack all samples of all runs belonging to a configuration.

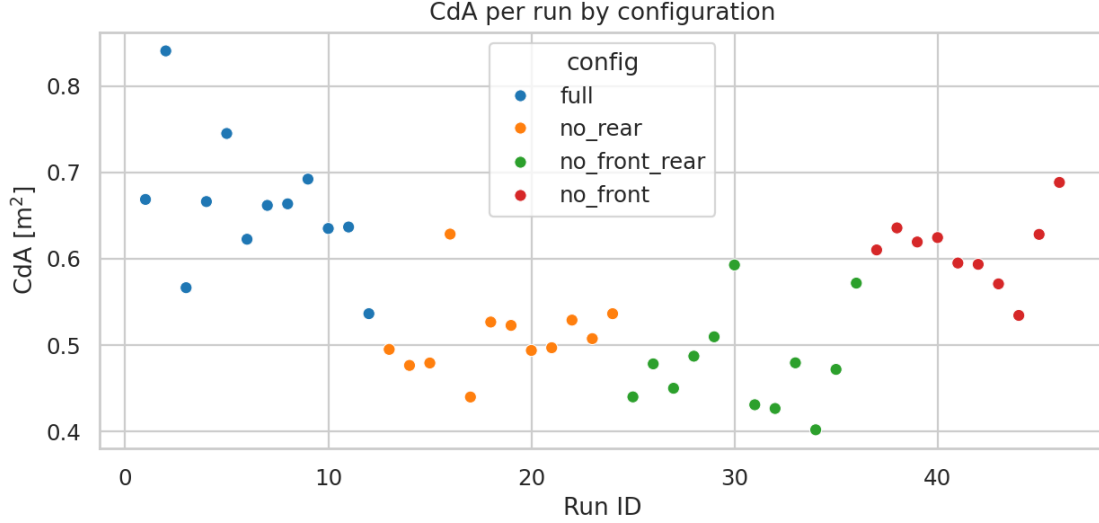


Figure 2: Approach 1: per-run C_dA values grouped by configuration.

- Fit one $k_{2,c}$ per configuration using robust regression.
- Motivation: maximize point count and reduce sensitivity to short noisy runs.

6 Outputs

Each approach writes:

- `Outputs/03_Drag_Calculation/<Approach>/csv/`: coefficients, per-config summaries, contributions, and a drag-force table.
- `Outputs/03_Drag_Calculation/<Approach>/plots/`: summary plots, and (for per-run modes) per-run coastdown fit plots.

7 Results (current Outputs/)

This section summarizes the results currently present in `Outputs/03_Drag_Calculation/`.

7.1 Uncertainty definition used in this report

The overview tables below report a “measurement uncertainty” per configuration:

- Approach 1: standard deviation across runs (`CdA_m2_std`, `Cr_std` from `drag_config_summary.csv`).
- Approach 2: standard deviation across paired runs.
- Approach 3: bootstrap standard deviation of the mean across runs (computed from Approach 1 per-run results and saved as `Outputs/03_Drag_Calculation/bootstrap_pooled_uncertainty`).

7.2 Baseline model results (Approaches 1–3)

Overview: C_dA

Overview: C_r

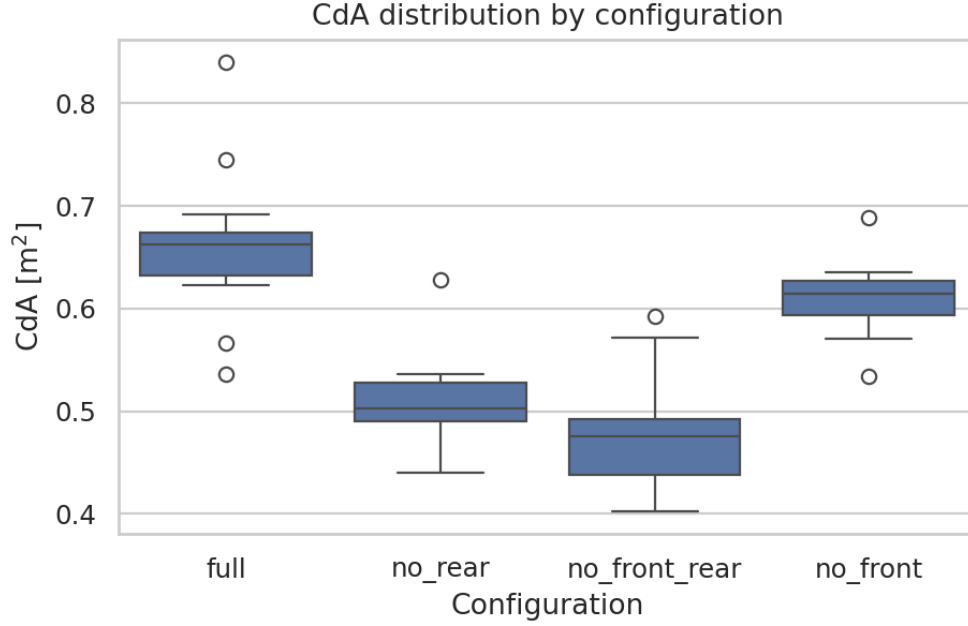


Figure 3: Approach 1: C_dA distribution per configuration (box plot).

Table 1: Baseline model: C_dA per configuration across Approaches 1–3 (mean/pooled value $\pm$ uncertainty).

Config	A1 (per-run)	A2 (paired)	A3 (pooled)
full	0.661 ± 0.078	0.661 ± 0.058	0.679 ± 0.022
no_rear	0.511 ± 0.046	0.511 ± 0.026	0.506 ± 0.014
no_front_rear	0.478 ± 0.057	0.478 ± 0.048	0.487 ± 0.015
no_front	0.610 ± 0.041	0.610 ± 0.039	0.647 ± 0.012

8 Reproducibility (example commands)

```
# Step 2 prerequisites:
# - drag intervals: Outputs/02_Run_Extraction/csv/drag_intervals.csv
# - per-run CSVs:   Outputs/02_Run_Extraction/csv/run_XXX.csv

# Baseline per-run fit
python Code/03_Drag_Calculation/Approach1_PerRun/run.py

# Paired opposite-direction averaging
python Code/03_Drag_Calculation/Approach2_PairedOpposites/run.py

# One pooled fit per configuration
python Code/03_Drag_Calculation/Approach3_PooledByConfig/run.py

# Optional: regenerate bootstrap uncertainty for pooled reporting
python Code/03_Drag_Calculation/bootstrap_pooled_uncertainty_from_runs.py
```

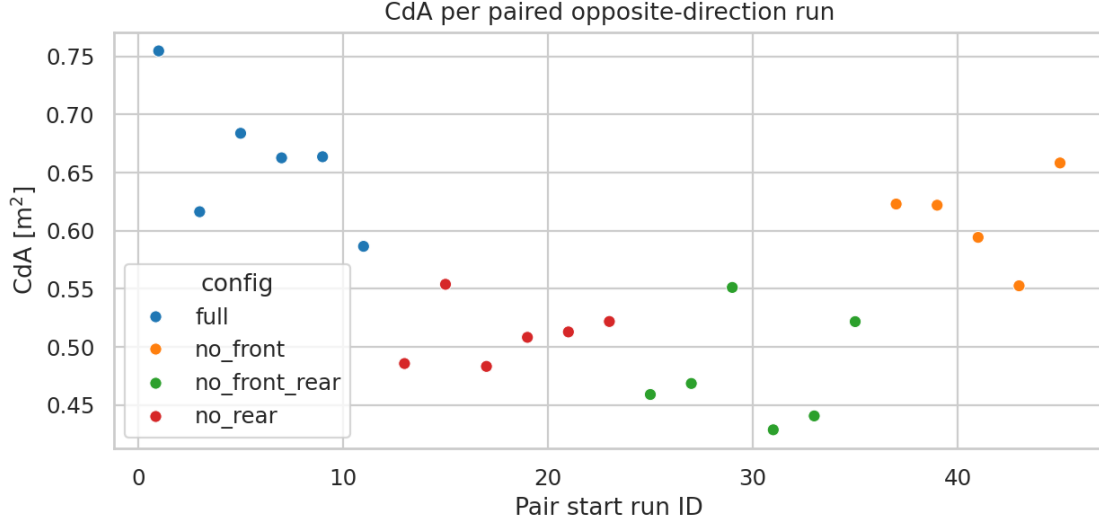


Figure 4: Approach 2: C_dA per paired runs, grouped by configuration.

Table 2: Baseline model: C_r per configuration across Approaches 1–3 (mean/pooled value $\pm$ uncertainty).

Config	A1 (per-run)	A2 (paired)	A3 (pooled)
full	0.0235 ± 0.0043	0.0235 ± 0.0014	0.0227 ± 0.0012
no_rear	0.0217 ± 0.0035	0.0217 ± 0.0015	0.0217 ± 0.0009
no_front_rear	0.0205 ± 0.0028	0.0205 ± 0.0013	0.0202 ± 0.0008
no_front	0.0217 ± 0.0033	0.0217 ± 0.0011	0.0201 ± 0.0010

9 Notes and limitations

- Grade and wind are not explicitly modeled inside a single run; pairing opposite directions reduces steady biases if the experiment is designed as out-and-back passes.
- All thresholds (minimum speed, filter gates) can be adjusted via CLI arguments of each approach.

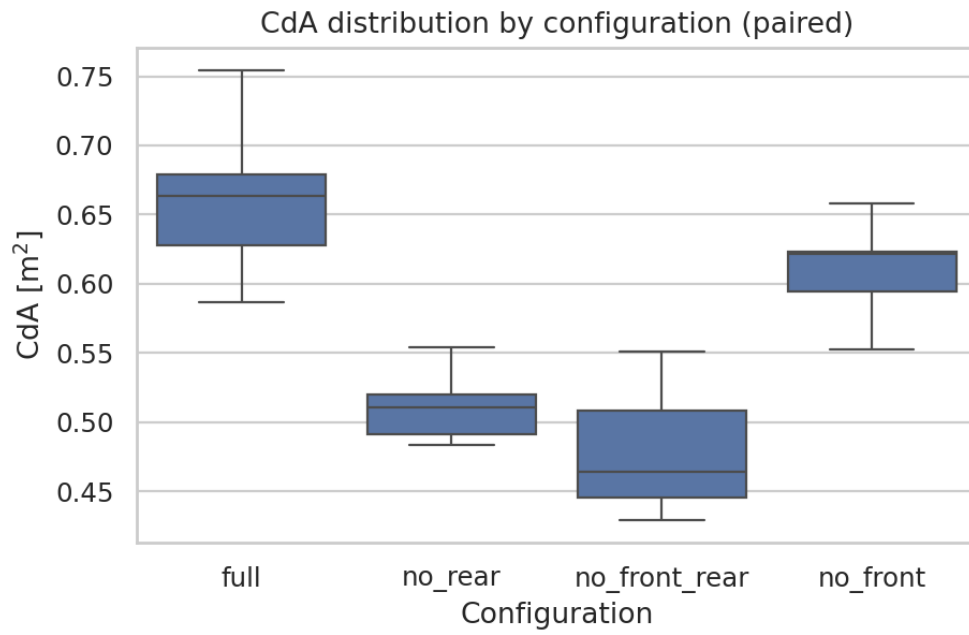


Figure 5: Approach 2: C_dA distribution per configuration after pairing.

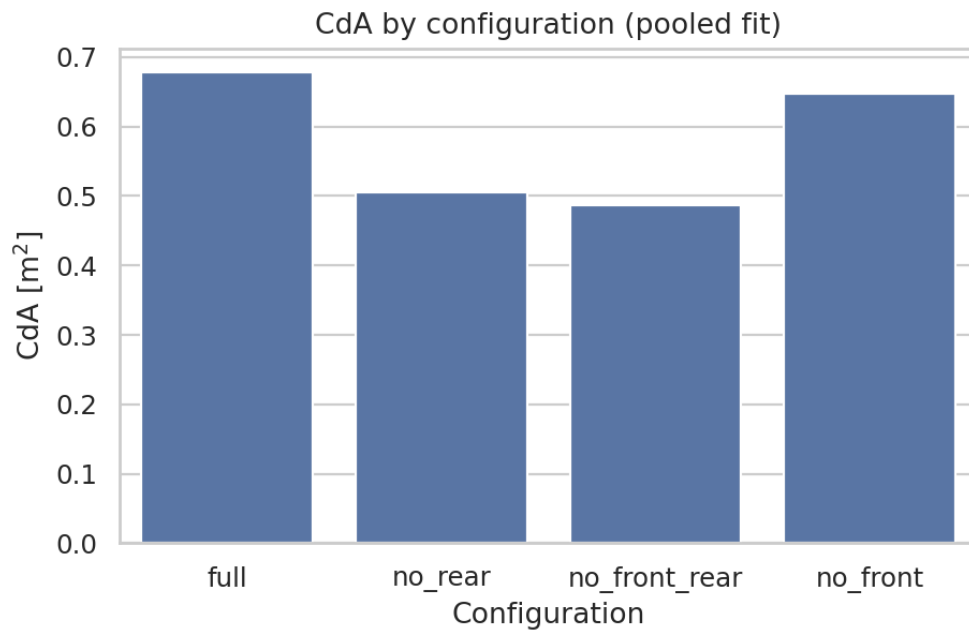


Figure 6: Approach 3: pooled C_dA estimate per configuration.